

Chapter 13. Reflection

The CLR knows a great deal about the types our programs define and use. It requires all assemblies to provide detailed metadata, describing each member of every type, including private implementation details. It relies on this information to perform critical functions, such as JIT compilation and garbage collection. However, it does not keep this knowledge to itself. The *reflection* API grants access to this detailed type information, so your code can discover everything that the runtime can see. Moreover, you can use reflection to make things happen. For example, a reflection object representing a method not only describes the method's name and signature, but it also lets you invoke the method. And you can go further still and generate code at runtime.

Reflection is particularly useful in extensible frameworks, because they can use it to adapt their behavior at runtime based on the structure of your code. For example, Visual Studio's Properties panel uses reflection to discover what public properties a component offers, so if you write a component that can appear on a design surface, such as a UI element, you do not need to do anything special to make its properties available for editing—Visual Studio will find them automatically.

NOTE

Many reflection-based frameworks that can automatically discover what they need to know also allow components to enrich that information explicitly. For example, although you don't need to do anything special to support editing in the Properties panel, you can customize the categorization, description, and editing mechanisms if you want to. This is normally achieved with *attributes*, which are the topic of [Chapter 14](#).

As you saw in the preceding chapter, .NET offers some deployment mechanisms that can limit the use of reflection. Trimming a self-contained deployment reduces its size by omitting code that your application does not use. This leaves out unused types entirely, but even with types you do use, individual members will be trimmed if your code doesn't use them directly or indirectly. Trimming removes type information as well as code, so reflection can provide an incomplete picture of a type in a trimmed application. Native AOT takes this further: it will often trim metadata even for members that are in use, retaining only executable code and the most basic information required to make GC work. This helps to reduce the size of the compiled output—if you won't be using reflection for the majority of the code in your project, there's no need to copy that information into the output. (This is possible because Native AOT never performs JIT compilation. Normally, .NET applications include full type information for all untrimmed code regardless of whether it will be used through reflection because the JIT compiler needs it.) In straightforward cases where trim analysis can determine that you will definitely be using reflection against particular members of a particular type, Native AOT will include the necessary metadata, but in general you should assume that reflection-based functionality might encounter limitations if you use Native AOT, and you will need to test carefully. (Be aware that when you enable trimming or Native AOT in a project, trimming occurs only when you ask the .NET SDK to *publish* your application—launching the code directly from the development environment typically doesn't apply either trimming or native code generation. This can make it easy to miss differences in behavior that occur once trimming has been applied.)

If you try to use a .NET runtime library feature that depends on reflection in an application that uses trimming, the compiler will warn you when you use it in a way that might not produce the behavior you wanted. Reflection-based methods in the runtime libraries have annotations that supply the trimmer with information that guides its operation, enabling it to determine either that it will need to include specific members, or that certain methods are essentially incompatible with trimming. In some cases you might need to add similar annotations to your own code to enable the analysis to fully understand your requirements. (The compiler will tell you when this is necessary.) Even then, there are some situations

that can defeat this analysis, so you may find that reflection-driven features don't initially work correctly when using trimming or Native AOT. Normally, adding your own annotations will enable you to deal with this. Alternatively, some parts of the runtime library that use reflection by default, such as the JSON serialization features described in [Chapter 15](#), offer reflection-free ways of using the APIs.

Not all .NET libraries have trimming annotations, because trimming is a relatively new feature. If you use a NuGet package that has no such annotations, trimming will have to rely entirely on analyzing the code to work out if and how it's using reflection. This can mean the warnings you get may be less helpful—you might get a warning when there isn't really a problem—or the trimmer may have to be conservative, and trim less code than it could.

Reflection Types

The reflection API defines various classes in the `System.Reflection` namespace. These classes have a structural relationship that mirrors the way that assemblies and the type system work. For example, a type's containing assembly is part of its identity, so the reflection class that represents a type (`Type`¹) has an `Assembly` property that returns its containing `Assembly` object. And you can navigate this relationship in both directions—you can discover all of the types in an assembly from the `Assembly` class's `DefinedTypes` property. An application that can be extended by loading plug-in DLLs would typically use this to find the types each plug-in provides.

[Figure 13-1](#) shows the reflection types that correspond to .NET types, their members, and the components that contain them. The arrows represent containment relationships. (As with assemblies and types, these are all navigable in both directions.)

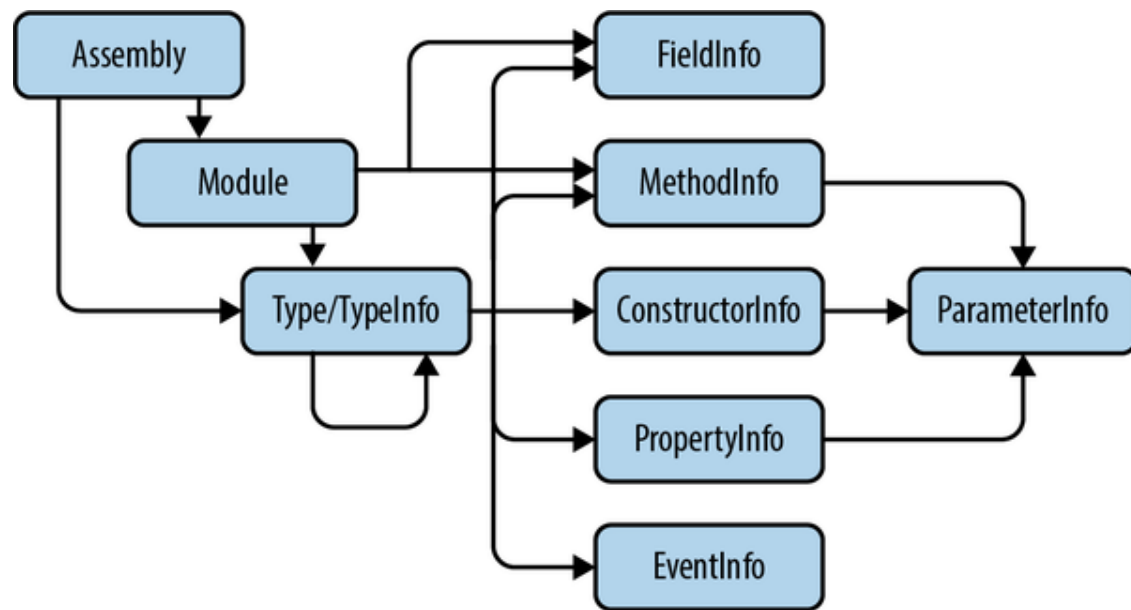


Figure 13-1. Reflection containment hierarchy

[Figure 13-2](#) illustrates the inheritance hierarchy for these types. This shows a couple of extra abstract types, `MemberInfo` and `MethodBase`, which are shared by various reflection classes that have a certain amount in common. For example, constructors and methods both have parameter lists, and the mechanism for inspecting these is provided by their shared base class, `MethodBase`. All members of types have certain common features, such as accessibility, so anything that is (or can be) a member of a type is represented in reflection by an object that derives from `MemberInfo`.

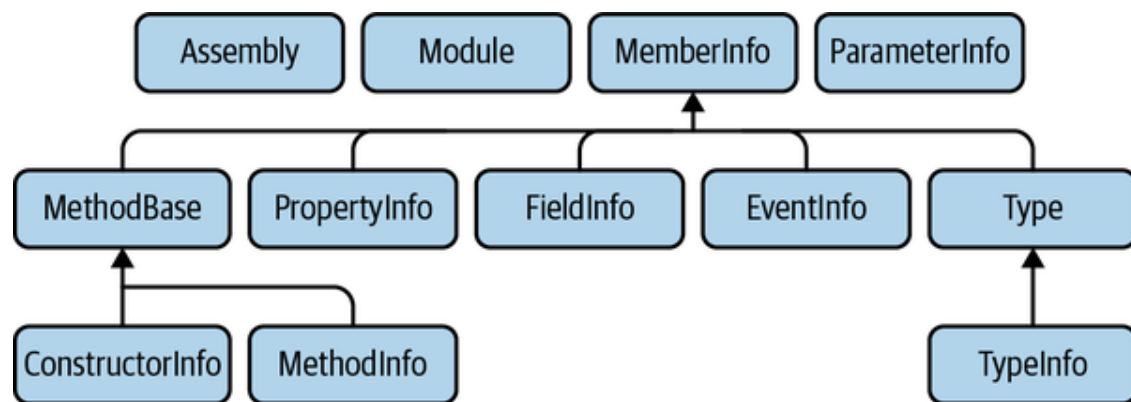


Figure 13-2. Reflection inheritance hierarchy

Assembly

The `Assembly` class represents, predictably enough, a single assembly. If you're writing a plug-in system, or some other sort of framework that needs to load user-supplied DLLs and use them (such as a unit test run-

ner), the `Assembly` type will be your starting point. As [Chapter 12](#) showed, the static `Assembly.Load` method takes an assembly name and returns the object for that assembly. (That method will load the assembly if necessary, but if it has already been loaded, it just returns a reference to the relevant `Assembly` object.) But there are some other ways to get hold of objects of this kind.

The `Assembly` class defines three context-sensitive static methods that each return an `Assembly`. The `GetEntryAssembly` method returns the object representing the EXE file containing your program's `Main` method. The `GetExecutingAssembly` method returns the assembly that contains the method from which you called it. `GetCallingAssembly` walks up the stack by one level and returns the assembly containing the code that called the method that called `GetCallingAssembly`.

NOTE

The JIT compiler's optimizations can sometimes produce surprising results with `GetExecutingAssembly` and `GetCallingAssembly`. Method inlining and tail call optimizations can both cause these methods to return the assembly for methods that are one stack frame farther back than you would expect. You can prevent inlining optimizations by annotating a method with the `MethodImplAttribute`, passing the `NoInlining` flag from the `MethodImplOptions` enumeration. (Attributes are described in [Chapter 14](#).) There's no way to disable tail call optimizations explicitly, but those will be applied only when a particular method call is the last thing a method does before returning.

`GetCallingAssembly` can be useful in diagnostic logging, because it provides information about the code that called your method. The `GetExecutingAssembly` method is less useful: you presumably already know which assembly the code will be in because you're the developer writing it. It may still be useful to get hold of the `Assembly` object for the component you're writing, but there are other ways. The `Type` object described in the next section provides an `Assembly` property. [Example 13-1](#) uses that to get the `Assembly` via the containing class. Empirically, this seems to be faster, which is not entirely surprising because it's doing less

work—both techniques need to retrieve reflection objects, but one of them also has to inspect the stack.

Example 13-1. Obtaining your own `Assembly` via a `Type`

```
class Program
{
    static void Main()
    {
        Assembly me = typeof(Program).Assembly;
        Console.WriteLine(me.FullName);
    }
}
```

If you want to use an assembly from a specific place on disk, you can use the `LoadFrom` method described in [Chapter 12](#). Alternatively, you can use the `System.Reflection.MetadataLoadContext` NuGet package's `MetadataLoadContext` class. This loads the assembly in such a way that you can inspect its type information, but no code in the assembly will execute, nor will any assemblies it depends on be loaded automatically. This is an appropriate way to load an assembly if you're writing a tool that displays or otherwise processes information about a component but does not want to run its code. There are a few reasons it can be important to avoid loading an assembly in the usual way with such a tool. Loading an assembly and inspecting its types can sometimes trigger the execution of code (such as static constructors) in that assembly. Also, if you load for reflection purposes only, the processor architecture is not significant, so you could load a 32-bit-only DLL into a 64-bit process, or you could inspect an ARM-only assembly in an x86 process.

Having obtained an `Assembly` from any of the aforementioned mechanisms, you can discover various things about it. The `FullName` property provides the display name, for example. Or you can call `GetName`, which returns an `AssemblyName` object, providing easy programmatic access to all of the components of the assembly's name.

You can retrieve a list of all of the other assemblies on which a particular `Assembly` depends by calling `GetReferencedAssemblies`. If you call this on an assembly you've written, it will not necessarily return all of the assemblies you can see in the Dependencies node in Visual Studio's Solution Explorer, because the C# compiler strips out unused references.

Assemblies contain types, so you can find `Type` objects representing those types by calling an `Assembly` object's `GetType` method, passing in the name of the type you require, including its namespace. This will return `null` if the type is not found, unless you call one of the overloads that additionally accept a `bool`—with these, passing `true` produces an exception if the type is not found. There's also an overload that takes two `bool` arguments, the second of which lets you pass `true` to request a case-insensitive search. All of these methods will return either `public` or `internal` types. You can also request a nested type, by specifying the name of the containing type, then a `+` symbol, then the nested type name. [Example 13-2](#) gets the `Type` object for a type called `Inside` nested inside a type called `ContainingType` in the `MyLib` namespace. This works even if the nested type is private.

Example 13-2. Getting a nested type from an assembly

```
Type? nt = someAssembly.GetType("MyLib.ContainingType+Inside");
```

The `Assembly` class also provides a `DefinedTypes` property that returns a collection containing a `TypeInfo` object for every type (top-level or nested) the assembly defines, and also `ExportedTypes`, which returns only public types, and it returns `Type` objects and not full `TypeInfo` objects. (The distinction is described in [“Type and TypeInfo”](#).) That will also include any `public` nested types. It will not include `protected` types nested inside `public` types, which is perhaps a bit surprising because such types are accessible from outside the assembly (albeit only to classes that derive from the containing type).

Besides returning types, `Assembly` can also create new instances of them with the `CreateInstance` method. If you pass just the fully qualified name of the type as a string, this will create an instance if the type is pub-

lic and has a no-arguments constructor. There's an overload that lets you work with nonpublic types and types with constructors that require arguments; however, it is rather more complex to use, because it also takes arguments that specify whether you want a case-insensitive match for the type name, along with a `CultureInfo` object that defines the rules to use for case-insensitive comparisons—different countries have different ideas about how such comparisons work. It also has arguments for controlling more advanced scenarios. However, you can pass `null` for most of these, as [Example 13-3](#) shows.

Example 13-3. Dynamic construction

```
object? o = asm.CreateInstance(  
    "MyApp.WithConstructor",  
    false,  
    BindingFlags.Public | BindingFlags.Instance,  
    null,  
    ["Constructor argument"],  
    null,  
    null);
```

This creates an instance of a type called `WithConstructor` in the `MyApp` namespace in the assembly to which `asm` refers. The `false` argument indicates that we want an exact match on the name, not a case-insensitive comparison. The `BindingFlags` indicate that we are looking for a public instance constructor. (See the sidebar [“BindingFlags.”](#)) The first `null` argument is where you could pass a `Binder` object, which allows you to customize the behavior when the arguments you have supplied do not exactly match the types of the required arguments. By leaving this out, I'm indicating that I expect the ones I've supplied to match exactly. (I'll get an exception if they don't.) The array argument contains the list of arguments I'd like to pass to the constructor—a single string, in this case. The penultimate `null` is where I'd pass a culture if I were using either case-insensitive comparisons or automatic conversions between numeric types and strings, but since I'm doing neither, I can leave it out. And the final argument once supported scenarios that have now been deprecated, so it should always be `null`.

BINDINGFLAGS

Many of the reflection APIs take an argument of the `BindingFlags` enumeration type to determine which members to return. For example, you can specify `BindingFlags.Public` to indicate that you want only public members or types, or `BindingFlags.NonPublic` to indicate that you want only items that are not public, or you can combine both flags to indicate that you'd like either.

Be aware that it's possible to specify combinations that will return nothing. When working with members, you must include either `BindingFlags.Instance`, `BindingFlags.Static`, or both, for example, because all type members are one or the other (likewise for `BindingFlags.Public` and `BindingFlags.NonPublic`).

Often, methods that can accept `BindingFlags` offer an overload that does not. This typically defaults to specifying public members, both instance and static (i.e., `BindingFlags.Public | BindingFlags.Static | BindingFlags.Instance`).

`BindingFlags` defines numerous options, but not all are applicable in every scenario. For example, it defines a `FlattenHierarchy` value, which is used for reflection APIs that return type members: if this flag is present, members defined by the base class will be considered, as well as those defined by the class specified. This option is not applicable to `Assembly.CreateInstance` because you cannot use a base class constructor directly to construct a derived type.

Module

[Figure 13-1](#) shows `Assembly` as a container of `Module` objects. As [Chapter 12](#) discussed, .NET Framework supports splitting the contents of one assembly across multiple files (*modules*), but this rarely used feature is not supported in .NET. In most cases, you can ignore the `Module` type—you can normally do everything you need with the other types in the reflection API. One exception is that the APIs for generating code at runtime require you to identify which module should contain the generated code,

even when you're creating just one module. (.NET's APIs for generating code at runtime are beyond the scope of this book.)

The `Module` class provides one other service: surprisingly, it defines `GetField`, `GetFields`, `GetMethod`, and `GetMethods` properties. These provide access to globally scoped methods and fields. You never see these in C#, because the language requires all fields and methods to be defined within a type, but the CLR allows globally scoped methods and fields, so the reflection API has to be able to present them. These are exposed through `Module`, and not `Assembly`, so even in modern .NET's one-module-per-assembly world, you can only get to them through the `Module` type. You can retrieve that from an `Assembly` object's `Modules` property, or you can use any of the API types described in the following sections that derive from `MemberInfo`. (Figure 13-2 shows which types do so.) This defines a `Module` property that returns the `Module` in which the relevant member is defined.

MemberInfo

Like all the classes I'm describing in this section, `MemberInfo` is abstract. However, unlike the rest, it does not correspond to one particular feature of the type system. It is a shared base class providing common functionality for all of the types that represent items that can be members of other types. So this is the base class of `ConstructorInfo`, `MethodInfo`, `FieldInfo`, `PropertyInfo`, `EventInfo`, and `Type`, because all of those can be members of other types. In fact, in C#, all except `Type` are *required* to be members of some other type (although, as you just saw in the preceding section, some languages allow methods and fields to be scoped to a module instead of a type).

`MemberInfo` defines common properties required by all type members. There's a `Name` property, of course, and also a `DeclaringType`, which refers to the `Type` object for the item's containing type; this returns `null` for nonnested types and module-scoped methods and fields. `MemberInfo` also defines a `Module` property that refers to the containing module, regardless of whether the item in question is module-scoped or a member of a type.

As well as `DeclaringType`, `MemberInfo` defines a `ReflectedType`, which indicates the type from which the `MemberInfo` was retrieved, which won't always be the declaring type when inheritance is involved. This should be an obscure area of the reflection API that could mostly be ignored, but unfortunately it has an unhelpful consequence. [Example 13-4](#) shows the problem.

Example 13-4. `DeclaringType` versus `ReflectedType`

```
class Base
{
    public void Foo()
    {
    }
}

class Derived : Base
{
}

class Program
{
    static void Main()
    {
        MemberInfo bf = typeof(Base).GetMethod("Foo")!;
        MemberInfo df = typeof(Derived).GetMethod("Foo")!;

        Console.WriteLine("Base    Declaring: {0}, Reflected: {1}",
                           bf.DeclaringType, bf.ReflectedType);
        Console.WriteLine("Derived Declaring: {0}, Reflected: {1}",
                           df.DeclaringType, df.ReflectedType);
        Console.WriteLine("Same:    {0}", bf == df);
    }
}
```

This gets the `MethodInfo` for the `Base.Foo` and `Derived.Foo` methods. (`MethodInfo` derives from `MemberInfo`.) These are just different ways of describing the same method—`Derived` does not define its own `Foo`, so it inherits the one defined by `Base`. However, the program produces this output:

```
Base      Declaring: Base, Reflected: Base
Derived Declaring: Base, Reflected: Derived
Same:     False
```

When retrieving the information for `Foo` via the `Base` class's `Type` object, the `DeclaringType` and `ReflectedType` are, unsurprisingly, both `Base`. However, when we retrieve the `Foo` method's information via the `Derived` type, the `DeclaringType` tells us that the method is defined by `Base`, while the `ReflectedType` tells us that we obtained this method via the `Derived` type.

WARNING

Because a `MemberInfo` remembers which type you retrieved it from, comparing two `MemberInfo` objects is not a reliable way to detect whether they refer to the same thing. When [Example 13-4](#) compares `bf` and `df`, the result is `false` despite the fact that they both refer to `Base.Foo`. If you had been unaware of the `ReflectedType` property, you might not have expected this behavior.

Slightly surprisingly, `MemberInfo` does not provide any information about the visibility of the member it describes. This may seem odd, because in C#, all of the constructs that correspond to the types that derive from `MemberInfo` (such as constructors, methods, or properties) can be prefixed with `public`, `private`, etc. The reflection API does make this information available but not through the `MemberInfo` base class. This is because the CLR handles visibility for certain member types a little differently from how C# presents it. From the CLR's perspective, properties and events do not have an accessibility of their own. Instead, their accessibility is managed at the level of the individual methods. This enables a property's `get` and `set` to have different accessibility levels, and likewise for an event's accessors. We can control property accessor accessibility independently in C# if we want to, but where C# misleads us is that it lets us specify a single accessibility level for the entire property. This is really just shorthand for setting both accessors to the same level. The confusing part is that it lets us specify the accessibility for the property and

then a different accessibility for one of the members, as [Example 13-5](#) does.

Example 13-5. Property accessor accessibility

```
public int Count
{
    get;
    private set;
}
```

This is a bit misleading because, despite how it looks, that `public` accessibility does not apply to the whole property. This property-level accessibility simply tells the compiler what to use for accessors that don't specify their own accessibility level. The first version of C# required both property accessors to have the same accessibility, so it made sense to state it for the whole property. (It still has an equivalent restriction for events.) But this was an arbitrary restriction—the CLR has always allowed each accessor to have a different accessibility. C# now supports this, but because of the history, the syntax for exploiting this is deceptively asymmetric. From the CLR's point of view, [Example 13-5](#) just says to make the `get` `public` and the `set` `private`. [Example 13-6](#) would be a better representation of what's really going on.

Example 13-6. How the CLR sees property accessibility

```
// Won't compile but arguably should
int Count
{
    public get;
    private set;
}
```

But we can't write it that way, because C# demands that the accessibility for the more visible of the two accessors be stated at the property level. This makes the syntax simpler when both properties have the same accessibility, but it makes things a bit weird when they're different. Moreover, the syntax in [Example 13-5](#) (i.e., the syntax the compiler actu-

ally supports) makes it look like we should be able to specify accessibility in three places: the property and both of the accessors. The CLR does not support that, so the compiler will produce an error if you try to specify accessibility for both of the accessors. So there is no accessibility for the property or event itself. (Imagine if there were—what would it even mean if a property had `public` accessibility but its `get` were `internal` and its `set` were `private`?) Consequently, not everything that derives from `MemberInfo` has a particular accessibility, so the reflection API provides properties representing accessibility farther down in the class hierarchy.

Type and TypeInfo

The `Type` class represents a particular type. It is more widely used than any of the other classes in this chapter, which is why it alone lives in the `System` namespace while the rest are defined in `System.Reflection`. It's the easiest to get hold of because C# has an operator designed for just this job: `typeof`. I've shown this in a few examples already, but [Example 13-7](#) shows it in isolation. As you can see, you can use either a built-in name, such as `string`, or an ordinary type name, such as `IDisposable`. You could also include the namespace, but that's not necessary when the type's namespace is in scope.

Example 13-7. Getting a `Type` with `typeof`

```
Type stringType = typeof(string);
Type disposableType = typeof(IDisposable);
```

Also, as I mentioned in [Chapter 6](#), the `System.Object` type (or `object`, as we usually write it in C#) provides a `GetType` instance method that takes no arguments. You can call this on any reference type variable to retrieve the type of the object that variable refers to. This will not necessarily be the same type as the variable itself, because the variable may refer to an instance of a derived type. You can also call this method on any value type variable, and because value types do not support inheritance, it will always return the `Type` object for the variable's static type.

So all you need is an object, a value, or a type identifier (such as `string`), and it is trivial to get a `Type` object. And, there are many other places `Type` objects can come from.

In addition to `Type`, we also have `TypeInfo`. This was introduced in early versions of .NET (called .NET Core at the time, to distinguish it from the .NET Framework) with the intention of enabling `Type` to serve purely as a lightweight identifier, and for `TypeInfo` to be the mechanism by which you reflect against a type. This was a departure from how `Type` had always worked in .NET Framework, where it performs both roles. This dual role was arguably a mistake because if you only need an identifier, `Type` is unnecessarily heavyweight. .NET Core was originally envisaged as having a separate existence from .NET Framework with no need for strict compatibility, so it seemed to provide an opportunity to fix historical design problems. However, once Microsoft made the decision that .NET Core would be the basis of all future versions of .NET, it became necessary to bring it back into line with how .NET Framework had always worked. However, by this time, .NET Framework had also introduced `TypeInfo`, and for a while, new type-level reflection features were added to that instead of `Type` to minimize incompatibilities with .NET Core 1. .NET Core 2.0 realigned with .NET Framework, but this meant that the split of functionality between `Type` and `TypeInfo` is now just an upshot of what was added when. `TypeInfo` contains members added during the brief period between its introduction and the decision to revert to the old way. In cases where you have a `Type` but you need to use a feature specific to `TypeInfo`, you can get this from a `Type` by calling `GetTypeInfo`.

As you've already seen, you can retrieve `Type` objects from an `Assembly`, either by name or as a comprehensive list. The reflection types that derive from `MemberInfo` also provide a reference to their containing type through `DeclaringType`. (`Type` derives from `MemberInfo`, so it also offers this property, which is relevant when dealing with nested types.)

You can also call the `Type` class's own static `GetType` method. If you pass just a namespace-qualified string, it will search for the named type in a system assembly called `mscorlib`, and also in the assembly from which you called the method. However, you can pass an *assembly-qualified*

name, which combines an assembly name and a type name. A name of this form starts with the namespace-qualified type name, followed by a comma and the assembly name. For example, this is the assembly-qualified name of the `System.String` class in .NET Framework 4.8.1 (split across two lines to fit in this book):

```
System.String, mscorlib, Version=4.0.0.0, Culture=neutral,  
PublicKeyToken=b77a5c561934e089
```

You can discover a type's assembly-qualified name through the `Type.AssemblyQualifiedName` property. Be aware that this won't always match what you asked for. If you pass the preceding type name into `Type.GetType` on .NET 8.0, it will work, but if you then ask the returned `Type` for its `AssemblyQualifiedName`, it will return this instead:

```
System.String, System.Private.CoreLib, Version=8.0.0.0, Culture=neutral,  
PublicKeyToken=7cec85d7bea7798e
```

The only reason it works when you pass either the first string or just `System.String` is because `mscorlib` still exists for backward compatibility purposes. I described this in the preceding chapter, but to summarize, in .NET Framework, the `mscorlib` assembly contains the core types of the runtime libraries, but in .NET, the code has moved elsewhere. `mscorlib` still exists, but it contains only type forwarding entries indicating which assembly each class now lives in. For example, it forwards `System.String` to its new home, which, at the time of this writing, is the `System.Private.CoreLib` assembly.

As well as the standard `MemberInfo` properties, such as `Module` and `Name`, the `Type` and `TypeInfo` classes add various properties of their own. The inherited `Name` property contains the unqualified name, so `Type` adds a `Namespace` property. All types are scoped to an assembly, so `Type` defines an `Assembly` property. (You could, of course, get there via `Module.Assembly`, but it's more convenient to use the `Assembly` property.) It also defines a `BaseType` property, although that will be `null` for

some types (e.g., nonderived interfaces and the type object for the `System.Object` class).

Since `Type` can represent all sorts of types, there are properties you can use to determine exactly what you've got: `IsArray`, `IsClass`, `IsEnum`, `IsInterface`, `IsPointer`, and `IsValueType`. (You can also get `Type` objects for non-.NET types in interop scenarios, so there's also an `IsCOMObject` property.) If it represents a class, there are some properties that tell you more about what kind of class you've got: `IsAbstract`, `IsSealed`, and `IsNested`. That last one is applicable to value types as well as classes.

`Type` also defines numerous properties providing information about the type's visibility. For nonnested types, `IsPublic` tells you whether it's `public` or `internal`, but things are more complex for nested types. `IsNestedAssembly` indicates an `internal` nested type, while `IsNestedPublic` and `IsNestedPrivate` indicate `public` and `private` nested types. Instead of the usual C-family "protected" terminology, the CLR uses the term *family*, so we have `IsNestedFamily` for `protected`, `IsNestedFamORAssem` for `protected internal`, and `IsNestedFamANDAssem` for `protected private`.

NOTE

There is no `IsRecord` property. As far as the runtime is concerned, record types are classes or structs. Records are a feature of the C# type system but not of the .NET runtime's type system, the CTS. Reflection is a runtime feature, so it presents the CTS perspective.

The `TypeInfo` class also provides methods to discover related reflection objects. (The properties in this paragraph are all defined on `TypeInfo`, not `Type`. As previously discussed, this is just an accident of when they were defined.) Most of these come in two forms: one where you want a complete list of all the items of the specified kind and one where you know the name of the thing you're looking for. For example, we have `DeclaredConstructors`, `DeclaredEvents`, `DeclaredFields`, `DeclaredMethods`, `DeclaredNestedTypes`, and `DeclaredProperties`

along with their counterparts, `GetDeclaredConstructor`, `GetDeclaredEvent`, `GetDeclaredField`, `GetDeclaredMethod`, `GetDeclaredNestedType`, and `GetDeclaredProperty`.

The `Type` class lets you discover type compatibility relationships. You can ask whether one type derives from another type by calling the type's `IsSubclassOf` method. Inheritance is not the only reason one type may be compatible with a reference of a different type—a variable whose type is an interface can refer to an instance of any type that implements that interface, regardless of its base class. The `Type` class therefore offers a more general method called `IsAssignableFrom`, shown in [Example 13-8](#), which tells you whether an implicit reference conversion exists.

Example 13-8. Testing type compatibility

```
Type stringType = typeof(string);
Type objectType = typeof(object);
Console.WriteLine(stringType.IsAssignableFrom(objectType));
Console.WriteLine(objectType.IsAssignableFrom(stringType));
```

This shows `False` and then `True`, because you cannot take a reference to an instance of type `object` and assign it into a variable of type `string`, but you can take a reference to an instance of type `string` and assign it into a variable of type `object`.

As well as telling you things about a type and its relationships to other types, the `Type` class provides the ability to use a type's members at runtime. It defines an `InvokeMember` method, the exact meaning of which depends on what kind of member you invoke—it could mean calling a method, or getting or setting a property or field, for example. Since some member types support multiple kinds of invocation (e.g., both get and set), you need to specify which particular operation you want.

[Example 13-9](#) uses `InvokeMember` to invoke a method identified by its name (the `member` string argument) on an instance of a type, also identified by name, that it instantiates dynamically. This illustrates how reflection can be used to work with types and members whose identities are not known until runtime.

Example 13-9. Invoking a method with InvokeMember

```
public static object? CreateAndInvokeMethod(
    string typeName, string member, params object[] args)
{
    Type t = Type.GetType(typeName)
        ?? throw new ArgumentException(
            $"Type {typeName} not found", nameof(typeName));
    object instance = Activator.CreateInstance(t)!;
    return t.InvokeMember(
        member,
        BindingFlags.Instance | BindingFlags.Public | BindingFlags.InvokeMethod,
        null,
        instance,
        args);
}
```

This example first creates an instance of the specified type—this uses a slightly different approach to dynamic creation than the one I showed earlier with `Assembly.CreateInstance`. Here I’m using `Type.GetType` to look up the type, and then I’m using a class I’ve not mentioned before, `Activator`. This class’s job is to create new instances of objects whose type you have determined at runtime. Its functionality overlaps somewhat with `Assembly.CreateInstance`, but in this case, it’s the most convenient way to get from a `Type` to a new instance of that type. Then I’ve used the `Type` object’s `InvokeMember` to invoke the specified method. As with [Example 13-3](#), I’ve had to specify binding flags to indicate what kind of member I’m looking for and also what to do with it—here I’m looking to call a method (as opposed to, say, setting a property value). As with [Example 13-3](#), the `null` argument is a place where I would have specified a `Binder` if I had wanted to support automatic coercion of the method argument types.

Generic Types

.NET’s support for generics complicates the role of the `Type` class. As well as representing an ordinary nongeneric type, a `Type` can represent a particular instance of a generic type (e.g., `List<int>`) but also an un-

bound generic type (e.g., `List<>`, although that's an illegal type identifier in all but one very specific scenario). [Example 13-10](#) shows how to obtain both kinds of `Type` objects.

Example 13-10. `Type` objects for generic types

```
Type bound = typeof(List<int>);  
Type unbound = typeof(List<>);
```

The `typeof` operator is the only place in which you can use an unbound generic type identifier in C#—in all other contexts, it would be an error not to supply type arguments. By the way, if the type takes multiple type arguments, you must provide commas—for example, `typeof(Dictionary<,>)`. This is necessary to avoid ambiguity when there are multiple generic types with the same names, distinguished only by the number of type parameters (also known as the *arity*)—for example, `typeof(Func<,>)` versus `typeof(Func<,,>)`. You cannot specify a partially bound generic type. For example, `typeof(Dictionary<string,>)` would fail to compile.

You can tell when a `Type` object refers to a generic type—the `IsGenericType` property will return `true` for both `bound` and `unbound` from [Example 13-10](#). You can also determine whether or not the type arguments have been supplied by using the `IsGenericTypeDefinition` property, which would return `false` and `true` for `bound` and `unbound`, respectively. If you have a bound generic type and you'd like to get the unbound type from which it was constructed, you use the `GetGenericTypeDefinition` method—calling that on `bound` would return the same type object that `unbound` refers to.

Given a `Type` object whose `IsGenericTypeDefinition` property returns `true`, you can construct a new bound version of that type by calling `MakeGenericType`, passing an array of `Type` objects, one for each type argument.

If you have a generic type, you can retrieve its type arguments from the `GenericTypeArguments` property. Perhaps surprisingly, this even works

for unbound types, although it behaves differently than with a bound type. If you get `GenericTypeArguments` from `bound` from [Example 13-10](#), it will return an array containing a single `Type` object, which will be the same one you would get from `typeof(int)`. If you get `unbound.GenericTypeArguments`, you will also get an array containing a single `Type`, but this time, it will be a `Type` object that does not represent a specific type—its `IsGenericParameter` property will be `true`, indicating that this represents a placeholder. Its name in this case will be `T`. In general, the name will correspond to whatever placeholder name the generic type chooses. For example, with `typeof(Dictionary<, >)`, you'll get two `Type` objects called `TKey` and `TValue`, respectively. You will encounter similar generic argument placeholder types if you use the reflection API to look up members of generic types. For example, if you retrieve the `MethodInfo` for the `Add` method of the unbound `List<>` type, you'll find that it takes a single argument of a type named `T`, which returns `true` from its `IsGenericParameter` property.

When a `Type` object represents an unbound generic parameter, you can find out whether the parameter is covariant or contravariant (or neither) through its `GenericParameterAttributes` method.

MethodBase, ConstructorInfo, and MethodInfo

Constructors and methods have a great deal in common. The same accessibility options are available for both kinds of members, they both have argument lists, and they can both contain code. Consequently, the `MethodInfo` and `ConstructorInfo` reflection types share a base class, `MethodBase`, which defines properties and methods for handling these common aspects.

To obtain a `MethodInfo` or `ConstructorInfo`, besides using the `Type` class properties I mentioned earlier, you can also call the `MethodBase` class's static `GetCurrentMethod` method. This inspects the calling code to see if it's a constructor or a normal method and returns either a `MethodInfo` or `ConstructorInfo` accordingly.

As well as the members it inherits from `MemberInfo`, `MethodBase` defines properties specifying the member's accessibility. These are similar in concept to those I described earlier for types, but the names are marginally different, because unlike `Type`, `MethodBase` does not define accessibility properties that make a distinction between nested and nonnested members. So with `MethodBase`, we find `IsPublic`, `IsPrivate`, `IsAssembly`, `IsFamily`, `IsFamilyOrAssembly`, and `IsFamilyAndAssembly` for `public`, `private`, `internal`, `protected`, `protected internal`, and `protected private`, respectively.

In addition to accessibility-related properties, `MethodBase` defines properties that tell you about aspects of the method, such as `IsStatic`, `IsAbstract`, `IsVirtual`, `IsFinal`, and `IsConstructor`.

There are also properties for dealing with generic methods.

`IsGenericMethod` and `IsGenericMethodDefinition` are the method-level equivalents of the type-level `IsGenericType` and `IsGenericTypeDefinition` properties. As with `Type`, there's a `GetGenericMethodDefinition` method to get from a bound generic method to an unbound one, and a `MakeGenericMethod` to produce a bound generic method from an unbound one. You can retrieve type arguments by calling `GetGenericArguments`, and as with generic types, this will return specific types when called on a bound method and will return placeholder types when used with an unbound method.

You can inspect the implementation of the method by calling

`GetMethodBody`. This returns a `MethodBody` object that provides access to the IL (as an array of bytes) and also to the local variable definitions used by the method.

The `MethodInfo` class derives from `MethodBase` and represents only methods (and not constructors). It adds a `ReturnType` property that provides a `Type` object indicating the method's return type. (There's a special system type, `System.Void`, whose `Type` object is used here when a method returns nothing.)

The `ConstructorInfo` class does not add any properties beyond those it inherits from `MethodBase`. It does define two read-only static fields,

though: `ConstructorName` and `TypeConstructorName`. These contain the strings `".ctor"` and `".cctor"`, respectively, which are the values you will find in the `Name` property for `ConstructorInfo` objects for instance and static constructors. As far as the CLR is concerned, these are the real names—although in C# constructors appear to have the same name as their containing type, that's true only in your C# source files, and not at runtime.

You can invoke the method or constructor represented by a `MethodInfo` or `ConstructorInfo` by calling the `Invoke` method. This does the same thing as `Type.InvokeMember`—[Example 13-9](#) used that to call a method. However, because `Invoke` is specialized for working with just methods and constructors, it's rather simpler to use. With a `ConstructorInfo`, you need to pass only an array of arguments. With `MethodInfo`, you also pass the object on which you want to invoke the method, or `null` if you want to invoke a static method. [Example 13-11](#) performs the same job as [Example 13-9](#) but using `MethodInfo`.

Example 13-11. Invoking a method

```
public static object? CreateAndInvokeMethod(
    string typeName, string member, params object[] args)
{
    Type t = Type.GetType(typeName)
        ?? throw new ArgumentException(
            $"Type {typeName} not found", nameof(typeName));
    MethodInfo m = t.GetMethod(member)
        ?? throw new ArgumentException(
            $"Method {member} not found", nameof(member));
    object instance = Activator.CreateInstance(t)!;
    return m.Invoke(instance, args);
}
```

For either methods or constructors, you can call `GetParameters`, which returns an array of `ParameterInfo` objects representing the method's parameters.

ParameterInfo

The `ParameterInfo` class represents parameters for methods or constructors. Its `ParameterType` and `Name` properties provide the basic information you'd see from looking at the method signature. It also defines a `Member` property that refers back to the method or constructor to which the parameter belongs. The `HasDefaultValue` property will tell you whether the parameter is optional, and if it is, `DefaultValue` provides the value to be used when the argument is omitted.

If you are working with members defined by unbound generic types, or with an unbound generic method, be aware that the `ParameterType` of a `ParameterInfo` could refer to a generic type argument, and not a real type. This is also true of any `Type` objects returned by the reflection objects described in the next three sections.

FieldInfo

`FieldInfo` represents a field in a type. You typically obtain it from a `Type` object with `GetField` or `GetFields`, or if you're using code written in a language that supports global fields, you can retrieve those from the containing `Module`.

`FieldInfo` defines a set of properties representing accessibility. These look just like the ones defined by `MethodBase`. Additionally, there's `FieldType`, representing the type a field can contain. (As always, if the member belongs to an unbound generic type, this might refer to a type argument rather than a specific type.) There are also some properties providing further information about the field, including `IsStatic`, `IsInitOnly`, and `IsLiteral`. These correspond to `static`, `readonly`, and `const` in C#, respectively. (Fields representing values in enumeration types will also return `true` from `IsLiteral`.)

`FieldInfo` defines `GetValue` and `SetValue` methods that let you read and write the value of the field. These take an argument specifying the instance to use, or `null` if the field is static. As with the `MethodBase` class's

Invoke , these do not do anything you couldn't do with the `Type` class's `InvokeMember` , but these methods are typically more convenient.

PropertyInfo

The `PropertyInfo` type represents a property. You can obtain these from the containing `Type` object's `GetProperty` or `GetProperties` methods. As I mentioned earlier, `PropertyInfo` does not define any properties for accessibility, because the accessibility is determined at the level of the individual get and set methods. You can retrieve those with the `GetMethod` and `SetMethod` methods, which both return `MethodInfo` objects.

Much like with `FieldInfo` , the `PropertyInfo` class defines `GetValue` and `SetValue` methods for reading and writing the value. Properties are allowed to take arguments—C# indexers are properties with arguments, for example. So there are overloads of `GetValue` and `SetValue` that take arrays of arguments. Also, there is a `GetIndexParameters` method that returns an array of `ParameterInfo` objects, representing the arguments required to use the property. The property's type is available through the `PropertyType` property.

EventInfo

Events are represented by `EventInfo` objects, which are returned by the `Type` class's `GetEvent` and `GetEvents` methods. Like `PropertyInfo` , this does not have any accessibility properties, because the event's add and remove methods each define their own accessibility. You can retrieve those methods with `GetMethod` and `RemoveMethod` , which both return a `MethodInfo` . `EventInfo` defines an `EventHandlerType` , which returns the type of delegate that event handlers are required to supply.

You can attach and remove handlers by calling the `AddEventHandler` and `RemoveEventHandler` methods. As with all other dynamic invocation, these just offer a more convenient alternative to the `Type` class's `InvokeMember` method.

Reflection Contexts

.NET has a feature called *reflection contexts*. These enable reflection to provide a virtualized view of the type system. By writing a custom reflection context, you can modify how types appear—you can cause a type to look like it has extra properties, or you can add to the set of attributes that members and parameters appear to offer. ([Chapter 14](#) will describe attributes.)

Reflection contexts are useful because they make it possible to write reflection-driven frameworks that enable individual types to customize how they are handled but without forcing every type that participates into providing explicit support. Prior to the introduction of custom reflection contexts in .NET Framework 4.5, this was handled with various ad hoc systems. Take the Properties panel in Visual Studio, for example. This can automatically display every public property defined by any .NET object that ends up on a design surface (e.g., any UI component you write). It's great to have automatic editing support even for components that do not provide any explicit handling for that, but components should have the opportunity to customize how they behave at design time.

Because the Properties panel predates .NET Framework 4.5, it uses one of the ad hoc solutions: the `TypeDescriptor` class. This is a wrapper on top of reflection, which allows any class to augment its design-time behavior by implementing `ICustomTypeDescriptor`. This enables a class to customize the set of properties it offers for editing and also to control how they are presented, even offering custom editing UIs. This is flexible, but has the downside of coupling the design-time code with the runtime code—components that use this model cannot easily be shipped without also supplying the design-time code. So Visual Studio introduced its own virtualization mechanisms for separating the two.

To avoid having each framework define its own virtualization system, custom reflection contexts add virtualization directly into the reflection API. If you want to write code that can consume type information provided by reflection but can also support design-time augmentation or modification of that information, it's no longer necessary to use some sort

of wrapper layer. You can use the usual reflection types described earlier in this chapter, but it's now possible to ask reflection to give you different implementations of these types, providing different virtualized views.

You do this by writing a custom reflection context that describes how you want to modify the view that reflection provides. [Example 13-12](#) shows a particularly boring type followed by a custom reflection context that makes that type look like it has a property.

Example 13-12. A simple type, enhanced by a reflection context

```
class NotVeryInteresting
{
}

class MyReflectionContext : CustomReflectionContext
{
    protected override IEnumerable<PropertyInfo> AddProperties(Type type)
    {
        if (type == typeof(NotVeryInteresting))
        {
            var fakeProp = CreateProperty(
                MapType(typeof(string).GetTypeInfo()),
                "FakeProperty",
                o => "FakeValue",
                (o, v) => Console.WriteLine($"Setting value: {v}"));

            return new[] { fakeProp };
        }
        else
        {
            return base.AddProperties(type);
        }
    }
}
```

Code that uses the reflection API directly will see the `NotVeryInteresting` type exactly as it is, with no properties. However, we can map that type through `MyReflectionContext`, as [Example 13-13](#) shows.

Example 13-13. Using a custom reflection context

```
var ctx = new MyReflectionContext();
TypeInfo mappedType = ctx.MapType(typeof(NotVeryInteresting).GetTypeInfo());

foreach (PropertyInfo prop in mappedType.DeclaredProperties)
{
    Console.WriteLine($"{prop.Name} ({prop.PropertyType.Name})");
}
```

The `mappedType` variable holds a reference to the resulting mapped type. It still looks like an ordinary reflection `TypeInfo` object, and we can iterate through its properties in the usual way with `DeclaredProperties`, but because we've mapped the type through my custom reflection context, we see the modified version of the type. This code's output will show that the type appears to define one property called `FakeProperty`, of type `string`.

Summary

The reflection API makes it possible to write code whose behavior is based on the structure of the types it works with. This might involve deciding which values to present in a UI grid based on the properties an object offers, or it might mean modifying the behavior of a framework based on what members a particular type chooses to define. For example, parts of the ASP.NET Core web framework will detect whether your code is using synchronous or asynchronous programming techniques and adapt appropriately. These techniques require the ability to inspect code at runtime,² which is what reflection enables. All of the information in an assembly required by the type system is available to our code.

Furthermore, you can present this through a virtualized view by writing a custom reflection context, making it possible to customize the behavior of reflection-driven code.

Code that inspects the structure of types to drive its behavior often needs additional information. For example, the `System.Text.Json` namespace includes types described in [Chapter 15](#) that can convert between .NET ob-

jects and JSON documents. These typically rely on reflection, but you can take more precise control over the purpose by supplying extra information in the form of *attributes*. These are the topic of the next chapter.

- 1 For historical reasons discussed later, a subset of this functionality is in a derived type called `TypeInfo`. But the base `Type` class is the one you most often encounter.
- 2 ASP.NET Core 8.0 has introduced an alternate mechanism that uses Roslyn, the C# compiler API, enabling it to inspect the code at build time instead. It generates code embedding the relevant information to avoid the need for reflection, enabling use of Native AOT. Reflection is still used by default if you don't target Native AOT.